

Informe de Normalización y Análisis de la Base de Datos

Verificación de la **Tercera Forma Normal (3FN)** • Tablas funcionales vs. sin uso • Recomendaciones

Aplicación: AJUPTELCBO v2.0 (Laravel 13.15.0 / PHP 8.4 / MySQL) • Motor: MySQL (InnoDB) • BD: `db_ajuptelcbo_new` • Fecha del análisis: 29/08/2026
Alcance: Solo análisis y diagnóstico. *No se modificó ningún archivo, tabla, dato ni configuración del proyecto.*
Versión del informe: v2.0 — estado *posterior* a la limpieza aplicada en la migración `2026_08_29_000000_drop_tablas_sin_uso` (44 → 31 tablas).

1. Resumen ejecutivo

31

Tablas existentes actualmente en la base de datos

27

Tablas funcionales en uso real (CRUD, consultas, agentes IA, auditoría, RBAC)

0

Tablas SIN ninguna funcionalidad (todas las 13 detectadas ya fueron eliminadas)

3

Tablas de infraestructura latente del framework (0 registros, para features inactivos)

1

Tabla de control interno (migrations)

Veredicto general de la normalización (3FN)

La base de datos está **correctamente estructurada a nivel de 1FN y 2FN** en todas sus tablas: toda columna es atómica, todas las tablas tienen clave primaria simple y no existen dependencias parciales. A nivel de **3FN se considera "CUMPLE PARCIALMENTE"** (no 100%): se detectaron **2 columnas redundantes** en tablas funcionales con datos reales (`asociados.tipo_asoc` y `familiares.pare_fami`). Una tercera columna que antes se consideraba redundante (`voceros.nombre_completo_busqueda`) en realidad es una **columna generada STORED GENERATED** mantenida automáticamente por MySQL, por lo que **no se cuenta como violación**. Ninguna de las redundancias provoca pérdida de datos; son costos de mantenimiento, coherencia y consistencia.

Hallazgo principal sobre uso funcional

Tras la limpieza ejecutada previamente (migración `2026_08_29_000000_drop_tablas_sin_uso`), **ya no quedan tablas sin ninguna funcionalidad**. El esquema quedaría así:

- **27 tablas funcionales:** el núcleo de negocio (`asociados`, `voceros`, `familiares`, `empleados`, `proveedores` y `catálogos`), más los módulos funcionales (`blog`, `base de conocimiento`, `trámites`, `configuración`, `agente IA`, `auditoría`, `bitácora`, `respaldos` y `RBAC`).
- **3 tablas de infraestructura latente** (`jobs` , `failed_jobs` , `model_has_permissions`): existen porque el framework/paquete las crea por defecto; están vacías por configuraciones actuales (`QUEUE_CONNECTION=sync` , asignación de permisos por rol y no por modelo). No deben eliminarse sin antes ajustar la configuración.
- **1 tabla de control interno:** `migrations` (49 registros, 30 lotes). Necesaria.

2. Metodología del análisis

Procedimiento aplicado (sin modificar nada)

- 1) Inventario del esquema MySQL completo actual: 31 tablas, columnas, tipos, nulabilidad, índices y **28 claves foráneas** (vistas `information_schema`).
- 2) Conteo exacto de registros por tabla (consulta `COUNT(*)` sobre cada una de las 31 tablas).
- 3) Lectura de las 49 migraciones aplicadas para determinar qué tablas declara el proyecto y cuáles se eliminaron en la limpieza.

- 4) Rastreo del uso real en el código: controladores (`app/Http/Controllers`), Livewire (`app/Livewire`), modelos (`app/Models` — 19 modelos, sin modelos huérfanos), rutas, jobs, servicios (`app/Services` , `app/Ai`), comandos (`app/Console`) y vistas Blade.
- 5) Verificación de columnas derivadas: `asociados.tipo_asoc` , `familiares.pare_fami` , `voceros.nombre_completo_busqueda` (identificando que esta última es `STORED GENERATED`).
- 6) Aplicación del criterio clásico de 1FN, 2FN y 3FN tabla por tabla.

3. Inventario y clasificación de uso de las tablas (estado actual)

Se listan las **31 tablas reales** de la base de datos con su estado funcional. **Registros** = filas actuales; **Uso** = evidencia de uso en el código de la aplicación.

TABLA	REGISTROS	ESTADO DE USO	MODELO / GESTOR	FUNCIÓN REAL
asociados	43	USADA	Asociado	Núcleo: socios de la asociación. CRUD completo, importación, consultas, informes, exportación.
voceros	16	USADA	Vocero	Voceros. CRUD, consultas vocero-asociado, exportación.
familiares	8	USADA	Familiare	Familiares de asociados. CRUD, consultas, reportes, exportación.
empleados	1	USADA	Empleado	Empleados internos. CRUD, exportación, restaurar.
proveedores	6	USADA	Proveedore	Proveedores. CRUD, exportación, restaurar.
cargos	10	USADA	Cargo	Catálogo de cargos para empleados. CRUD.
estadosciviles	7	USADA	Estadoscivile	Catálogo de estado civil (Soltero, Casado, Viudo...). CRUD.
situaciones	10	USADA	Situacione	Catálogo de situación social de los socios. CRUD.
zonas	23	USADA	Zona	Catálogo de zonas/parroquias de ubicación. CRUD.
tipos_asociado	5	USADA	TipoAsociado	Catálogo tipo de asociado. Relación Asociado→tipoAsociado , validación exists y consultas de información.
tipos_parentesco	4	USADA	TipoParentesco	Catálogo parentesco. Relación Familiare→tipoParentesco y consultas.
users	13	USADA	Usuario	Autenticación. FK asociados_id → asociados .
blog_posts	16	USADA	BlogPost	Blog público del sitio (CRUD en BlogController + panel admin).
knowledge_base	7	USADA	KnowledgeBase	Base de conocimiento para agentes IA. CRUD + lectura por AgentContextService y herramienta ConsultarBaseConocimiento .
procedures	4	USADA	Procedure	Trámites/solicitudes de los socios. CRUD, dashboard y consulta por agente IA.
association_settings	1	USADA	AssociationSetting	Configuración única de la asociación (contacto, redes, RIF). Vistas de contacto, cabecera y SendDocumentsJob .
audit_ia	11	USADA	AuditIa	Registro de auditoría de las interacciones del asistente IA (AgentChatController).
bitacora_accesos	91	USADA	BitacoraAcceso	Bitácora de inicio/cierre de sesión (AuthController).
backup_schedules	5	USADA	BackupSchedule	Programación de respaldos/restauraciones de la BD (DatabaseProtectionController + RunBackupScheduleCommand).
activity_log	149	USADA	Spatie Activitylog	Log de actividad (trait HasActivity + ActivityLogController).

TABLA	REGISTROS	ESTADO DE USO	MODELO / GESTOR	FUNCIÓN REAL
agent_conversations	40	USADA	Laravel AI	Conversaciones del agente IA (<code>continue()</code> / <code>continueLastConversation()</code>).
agent_conversation_messages	80	USADA	Laravel AI	Mensajes de las conversaciones del agente IA.
roles / permissions	4 / 58	USADA	Spatie Permission	RBAC del sistema (roles y permisos por paquete Spatie).
role_has_permissions	75	USADA	Spatie Permission	Pivote rol ↔ permiso.
model_has_roles	10	USADA	Spatie Permission	Asignación rol ↔ usuario (<code>assignRole()</code>).
password_reset_tokens	2	USADA	Laravel Auth	Token de restablecimiento de contraseña (flujo "olvidé contraseña").
jobs	0	INFRAESTRUCTURA	Laravel Queue	Cola de trabajos. Migrada por defecto; <code>QUEUE_CONNECTION=sync</code> → sin residencia hoy. El <code>Job SendDocumentsJob</code> se ejecuta síncrono.
failed_jobs	0	INFRAESTRUCTURA	Laravel Queue	Trabajos fallidos. Sin uso por la misma razón. En <code>.env.example</code> el proyecto prevé <code>QUEUE_CONNECTION=database</code> .
model_has_permissions	0	INFRAESTRUCTURA	Spatie Permission	Asignación directa permiso ↔ modelo; hoy los permisos se otorgan vía roles (<code>role_has_permissions</code>). Exigida por el paquete.
migrations	49	SISTEMA	Laravel	Control de versiones de migraciones (30 lotes). Necesaria.

4. Tablas que NO tienen ninguna funcionalidad en el proyecto

Resultado: 0 tablas sin funcionalidad — ya fueron eliminadas

En el análisis actual **ninguna tabla restante carece de funcionalidad**. La revisión previa (versión v1 de este informe) detectó **13 tablas sin ningún uso**; esas tablas se eliminaron mediante la migración `2026_08_29_000000_drop_tablas_sin_uso` (lote 30), ya aplicada. Las 13 eliminadas fueron:

TABLA ELIMINADA	REGISTROS	MOTIVO POR EL QUE NO TENÍA NINGUNA FUNCIÓN
estados	0	Catálogo geográfico vacío; la geolocalización real se hace por API externa (<code>GeoLocationService</code>). Ningún servicio/controlador la consultaba.
municipios	0	Idem; dependía de <code>estados</code> .
parroquias	0	Idem; dependía de <code>municipios</code> .
ciudades	0	Idem; su única referencia era el modelo muerto <code>Direccion</code> .
situaciones_sociales	0	Duplicaba el catálogo activo <code>situaciones</code> (10 registros, con CRUD).
tipos_estadocivil	0	Duplicaba el catálogo activo <code>estadosciviles</code> (7 registros, con CRUD).
tipos_documento	0	Catálogo del esquema "personas" v1.1 nunca conectado a la lógica de negocio.
tipos_docdigital	0	Idem, solo la referenciaba el modelo muerto <code>DocumentoDigital</code> .
tipos_formacont	0	Idem; solo existía la clase <code>TipoFormaCont</code> .
tipos_redes_sociales	0	Idem; solo la referenciaba el modelo muerto <code>RedSocial</code> .
tipos_entidad	0	Idem; solo la referenciaba el modelo muerto <code>EntidadRedSocial</code> .
tipos_contacto	0	Idem; solo la referenciaba el modelo muerto <code>Contacto</code> .
personal_access_tokens	0	Tabla de Sanctum para tokens de API; el proyecto no usa el paquete.

Casos aparte: infraestructura latente (sin funcionalidad de negocio, pero NO reclamada para eliminar)

- `jobs` y `failed_jobs` : tablas del framework con 0 registros porque `QUEUE_CONNECTION=sync` . Son necesarias si se activa la cola en base de datos (ya prevista en `.env.example` , `QUEUE_CONNECTION=database`).
- `model_has_permissions` : exigida por el paquete Spatie Permission; quedó vacía porque la asignación de permisos se hace por rol. Eliminarla sin configurar el paquete rompería la migración `create_permission_tables` .
- **Modelos del proyecto**: los 19 modelos en `app/Models` tienen tabla real y uso. Ya no existen modelos "sin tabla" ni modelos huérfanos de esquemas antiguos (se eliminaron en la limpieza junto con sus tablas).

5. Análisis de la normalización (1FN → 2FN → 3FN)

Conceptos aplicados (breve recordatorio)

- **1FN (Primera Forma Normal)**: valores atómicos, sin grupos repetidos y sin listas/JSON en columnas.
- **2FN (Segunda Forma Normal)**: cumple 1FN y todos los atributos no-clave dependen de la clave completa (sin dependencias parciales). Requiere clave primaria.
- **3FN (Tercera Forma Normal)**: cumple 2FN y no existen dependencias transitivas, es decir, un atributo no-clave no depende de otro atributo no-clave (o de un valor que proviene de otra tabla y se copia).

5.1 Análisis paso a paso de las tablas funcionales del núcleo

1 Paso 1 — Claves primarias y atómica (1FN)

Todas las tablas del núcleo (`asociados` , `voceros` , `familiares` , `empleados` , `proveedores` y los catálogos `cargos` , `estadosciviles` , `situaciones` , `zonas` , `tipos_asociado` , `tipos_parentesco`) tienen una **clave primaria simple** con valores que no dependen de atributos de negocio. Cada columna almacena un único valor atómico (CI, apellido, nombre, fecha, teléfono, correo). No hay columnas con listas separadas por coma ni grupos repetidos. **1FN CUMPLE**

2 Paso 2 — Verificación de 2FN (sin dependencia parcial)

Al ser claves primarias de un solo atributo, no es posible una dependencia parcial. Todos los atributos no-clave dependen funcionalmente de toda la clave. **2FN CUMPLE** . Las referencias a otras entidades se modelan por **clave foránea** (28 FKs verificadas), no duplicando el dato: p. ej. `asociados.estadosciviles_id` → `estadosciviles.id` , `asociados.zonas_id` → `zonas.id` , `asociados.voceros_id` → `voceros.id` , `asociados.situaciones_id` → `situaciones.id` , `asociados.id_tipo_asociado` → `tipos_asociado.id_tipo_asoc` , `familiares.asociados_id` → `asociados.id` , `familiares.id_parentesco` → `tipos_parentesco.id_parentesco` , `empleados.cargos_id` → `cargos.id` , `proveedores.zonas_id` → `zonas.id` , `users.asociados_id` → `asociados.id` .

3 Paso 3 — Verificación de 3FN (sin dependencias transitivas)

La mayoría de atributos dependen solo de la clave primaria. Sin embargo se detectaron **2 columnas redundantes** (ver detalle en la sección 6) que impiden decir que estas tablas cumplen 3FN al 100%:

- `asociados.tipo_asoc` → repite el valor del catálogo `tipos_asociado.nombre_tipo_asoc` (verificado: 43/43 filas coinciden). Dependencia transitiva `id_tipo_asociado` → `tipo_asoc` .
- `familiares.pare_fami` → repite el valor del catálogo `tipos_parentesco.nombre_tipo_paren` (verificado: 8/8 filas coinciden). Dependencia transitiva `id_parentesco` → `pare_fami` .
- `voceros.nombre_completo_busqueda` → columna **generada** (`STORED GENERATED`), calculada por MySQL a partir de `ape_voce` y `nom_voce` . Al ser mantenida por el motor, **no es una redundancia manual**: se considera correcta y no viola 3FN.

3FN CUMPLE PARCIALMENTE (2 casos puntuales de redundancia controlada).

5.2 Análisis paso a paso de los módulos funcionales

TABLA	1FN	2FN	3FN	DETALLE / OBSERVACIÓN DE NORMALIZACIÓN
<code>blog_posts</code>	OK	OK	OK	PK simple <code>id</code> , <code>slug</code> único. <i>Observación:</i> <code>category</code> (varchar) y <code>tag_label</code> (varchar) guardan clasificación como texto libre; si crecen, deberían normalizarse en tablas de

TABLA	1FN	2FN	3FN	DETALLE / OBSERVACIÓN DE NORMALIZACIÓN
				categorías/etiquetas. No viola 3FN hoy.
knowledge_base	OK	OK	OK	tipo es ENUM (valores fijos: clínica, funeraria, farmacia, hospital, otro). Sin dependencias transitivas.
procedures	OK	OK	OK	FK user_id → users correcta. estado (ENUM de flujo) y status (varchar activo/inactivo) tienen semánticas distintas; no son redundantes.
association_settings	OK	OK	OK	Fila única de configuración (singleton). Observación: las 5 redes sociales son columnas del mismo tipo (whatsapp, instagram, facebook, twitter_x, youtube); se normalizaría en una tabla hija, pero al ser singleton es aceptable.
backup_schedules	OK	OK	OK	FK created_by → users . Solo atributos propios del respaldo. status ENUM y type ENUM correctos.
audit_ia	OK	OK	OK	PK propia id_log_ia , FK id_usuario → users . Tabla de log, correcta.
bitacora_accesos	OK	OK	OK	PK propia id_registro , FK id_usuario → users . Log, correcta.
activity_log	OK	OK	OK	Estructura polimórfica (subject/causer type+id) del paquete Spatie. Normalizada para su fin.
agent_conversations / agent_conversation_messages	OK	OK	OK	PK tipo UUID (varchar 36), FKs a users y a conversación. Estructura 1→N correcta. Relación conv↔mensajes normalizada.
users	OK	OK	OK	PK id , email UNIQUE, FK opcional asociados_id . Cumple 3FN.
roles , permissions , pivotes Spatie	OK	OK	OK	Tablas pivote con PKs compuestas (role_id+permission_id); todos los atributos dependen de la clave completa. model_has_roles / model_has_permissions : morfos estándar del paquete. Cumplen 3FN.
password_reset_tokens	OK	OK	OK	PK = email (identidad natural de autenticación). Cumple 3FN.

5.3 Análisis de las tablas de infraestructura latente

TABLA	1FN	2FN	3FN	DETALLE
jobs	EXCEP.	OK	OK	payload es JSON (no atómico), pero es la convención oficial de Laravel para colas; se acepta como excepción de 1FN. Estructura estándar del framework. 0 registros (cola síncrona).
failed_jobs	EXCEP.	OK	OK	Misma excepción de 1FN por payload / exception longtext. Periférica; 0 registros.
model_has_permissions	OK	OK	OK	PK compuesta (permission_id, model_type, model_id); dependencia total de la clave. Exigida por Spatie. 0 registros.
migrations	OK	OK	OK	Control interno. 49 registros en 30 lotes.

6. Hallazgos de normalización y diseño

6.1 Columnas redundantes / derivadas en tablas funcionales (violaciones de 3FN)

COLUMNA	DEPENDENCIA TRANSITIVA / DERIVACIÓN	EVIDENCIA VERIFICADA	IMPACTO
asociados.tipo_asoc	Dato copiado del catálogo: id_tipo_asociado → tipos_asociado.nombre_tipo_asoc = tipo_asoc	43/43 registros coinciden con el catálogo; el propio AsociadoRequest vuelve a resolver tipo_asoc → id_tipo_asociado al guardar.	Riesgo de inconsistencia si cambia el catálogo; redundancia que afecta al dashboard (GROUP BY tipo_asoc).
familiares.pare_fami	Dato copiado del catálogo: id_parentesco →	8/8 registros coinciden; en InfAsociadoConsultaController se usa el catálogo y pare_fami solo como respaldo (??).	Misma redundancia; existen dos

COLUMNA	DEPENDENCIA TRANSITIVA / DERIVACIÓN	EVIDENCIA VERIFICADA	IMPACTO
	tipos_parentesco.nombre_tipo_paren = pare_fami		fuentes para el mismo dato.
voceros.nombre_completo_busqueda	Valor derivado de ape_voce + nom_voce .	Columna GENERATED ALWAYS AS (ucase(concat(coalesce(trim(ape_voce), ''), ', ', coalesce(trim(nom_voce), '')))) STORED , con índice para el buscador de voceros.	Gestionada automáticamente por MySQL (STORED); no hay que mantenerla manualmente. Se considera correcta y NO una violación de 3FN.

6.2 Catálogos duplicados / huérfanos (a nivel de esquema)

Ya no existen. Los catálogos duplicados/huérfanos (tipos_estadocivil frente a estadosciviles , situaciones_sociales frente a situaciones , además de la geografía y los tokens de Sanctum) fueron eliminados. Hoy cada catálogo tiene una sola representación: estadosciviles , situaciones , zonas , cargos , tipos_asociado , tipos_parentesco .

- 6.3 Otros puntos de diseño relevantes (sin violar 3FN)
- **Duplicación estructural de "persona":** asociados , voceros , familiares y empleados comparten ~20 columnas idénticas (CI, apellido, nombre, nacimiento, estado civil, domicilio, zona, teléfonos, email, ingreso, observaciones, foto, status, género, elim_log, timestamps). No viola 3FN por tabla, pero indica que podría modelarse una superclase personas con herencia por rol.
 - **Doble control de estado:** casi todas las tablas usan status (varchar) y elim_log (varchar) como marcadores de activo/eliminado, sin ENUM ni booleano. Funciona, pero añade valores mágicos de texto.
 - **Columnas con valores por defecto no nulos pero de texto:** p. ej. foto_asoc y tipo_asoc con NOT NULL DEFAULT '' . Preferible NULL cuando no existe el valor.
 - **PKs no uniformes en catálogos:** tipos_asociado.id_tipo_asoc y tipos_parentesco.id_parentesco usan nombres de PK distintos al estándar id , resto de catálogos. Coherentes internamente pero no uniformes.
 - **Índices de unicidad limitados:** solo users.email y blog_posts.slug tienen restricción UNIQUE; las identidades naturales de negocio (CI, RIF) no tienen índice único.

7. Recomendaciones (solo propuesta — NO aplicadas)

- Sobre la normalización 3FN — prioridad alta
- **Eliminar asociados.tipo_asoc** y resolver el tipo siempre desde id_tipo_asociado → tipos_asociado (JOIN). Ajustar el dashboard que hace GROUP BY tipo_asoc a un GROUP BY con JOIN . Nota: el código ya resuelve el catálogo en AsociadoRequest y en las consultas.
 - **Eliminar familiares.pare_fami** y usar solo id_parentesco → tipos_parentesco (el código ya prefiere el catálogo con fallback ?? \$familiar->pare_fami).
 - **Mantener voceros.nombre_completo_busqueda** tal cual: al ser una columna generada STORED GENERATED , MySQL la mantiene sin intervención manual; es la forma recomendada de acelerar búsquedas sin romper 3FN.
 - Migrar status / elim_log a BOOLEAN o ENUM con valores explícitos, y dejar NULL donde no exista el dato (foto, fechas, etc.).

- Acerca de la estructura y consistencia — prioridad media
- Evaluar una superclase personas para los 4 actores que comparten columnas (asociados, voceros, familiares, empleados), solo si se planifica una refactorización mayor.

- Uniformar los nombres de las PK de catálogos (`id_tipo_asoc` , `id_parentesco`) hacia el estándar `id` si se consolida el esquema.
- Normalizar `blog_posts.category / tag_label` a tablas de catálogo si el blog crece.

Sobre restricciones e índices — prioridad baja

- Añadir **índices únicos** en las identidades naturales de negocio: `ci_asoc` , `ci_fami` , `ci_voce` , `ci_emp` , `rif_prov` , `cod_prov` , `email_asoc` , `email_emp` .
- Si se activa la cola en base de datos (`QUEUE_CONNECTION=database`), `jobs / failed_jobs` pasarían a ser funcionales sin cambios de esquema.
- Mantener `model_has_permissions` y `migrations` sin cambios: son requeridas por el framework/paquete.

8. Conclusión

Lado positivo: el esquema activo está bien diseñado. Hay claves primarias en todas las tablas, uso correcto y extendido de claves foráneas (28 FKs verificadas), valores atómicos y ausencia de dependencias parciales. Las tablas que realmente funcionan cumplen **1FN y 2FN al 100%** y, con 2 excepciones puntuales (`tipo_asoc` , `pare_fami`), también 3FN.

Estado de uso: de las 31 tablas, **27 son funcionales**, 3 son infraestructura latente del framework (`jobs` , `failed_jobs` , `model_has_permissions`) y 1 es de control interno (`migrations`). **No queda ninguna tabla sin funcionalidad:** las 13 tablas muertas del esquema heredado (geografía, catálogos duplicados, Sanctum) fueron eliminadas en la migración `2026_08_29_000000_drop_tablas_sin_uso` , que ya está aplicada (lote 30).

Cumplimiento 3FN global: la base de datos **cumple 3FN parcialmente** por 2 columnas redundantes (`asociados.tipo_asoc` y `familiares.pare_fami`). Ninguna de estas redundancias provoca pérdida o corrupción de datos; son costos de mantenimiento y riesgo futuro de inconsistencia. Su eliminación (resolviendo siempre por catálogo con JOIN) llevaría el esquema a un cumplimiento total de 3FN.

9. Anexos

A. Relaciones (claves foráneas) existentes — 28 FKs

TABLA HIJA	COLUMNA FK	→ TABLA PADRE	COLUMNA
agent_conversations	user_id	users	id
agent_conversation_messages	conversation_id	agent_conversations	id
agent_conversation_messages	user_id	users	id
asociados	estadosciviles_id	estadosciviles	id
asociados	id_tipo_asociado	tipos_asociado	id_tipo_asoc
asociados	situaciones_id	situaciones	id
asociados	voceros_id	voceros	id
asociados	zonas_id	zonas	id
audit_ia	id_usuario	users	id
backup_schedules	created_by	users	id
bitacora_accesos	id_usuario	users	id
empleados	cargos_id	cargos	id
empleados	estadosciviles_id	estadosciviles	id

TABLA HIJA	COLUMNA FK	→ TABLA PADRE	COLUMNA
empleados	zonas_id	zonas	id
familiares	asociados_id	asociados	id
familiares	estadosciviles_id	estadosciviles	id
familiares	id_parentesco	tipos_parentesco	id_parentesco
familiares	situaciones_id	situaciones	id
familiares	zonas_id	zonas	id
model_has_permissions	permission_id	permissions	id
model_has_roles	role_id	roles	id
procedures	user_id	users	id
proveedores	zonas_id	zonas	id
role_has_permissions	permission_id	permissions	id
role_has_permissions	role_id	roles	id
users	asociados_id	asociados	id
voceros	estadosciviles_id	estadosciviles	id
voceros	zonas_id	zonas	id

B. Evidencias de datos consultados (análisis del 29/08/2026)

COMPROBACIÓN	RESULTADO
asociados.tipo_asoc vs tipos_asociado.nombre_tipo_asoc	43/43 filas idénticas → columna redundante (violación parcial de 3FN).
familiares.pare_fami vs tipos_parentesco.nombre_tipo_paren	8/8 filas idénticas → columna redundante (violación parcial de 3FN).
voceros.nombre_completo_busqueda	Columna STORED GENERATED (EXTRA en information_schema) → gestionada por MySQL, no se cuenta como redundancia.
Tablas sin uso tras limpieza	0. Las 13 detectadas fueron eliminadas por la migración 2026_08_29_000000_drop_tablas_sin_uso (lote 30).
Infraestructura de cola	QUEUE_CONNECTION=sync en .env (y database en .env.example) → jobs =0, failed_jobs =0.
RBAC	role_has_permissions =75, model_has_roles =10, model_has_permissions =0 (permisos otorgados por rol).
Migraciones	49 aplicadas en 30 lotes; lote 30 = limpieza de tablas sin uso.

Informe generado de forma automática a partir del análisis estático del código, las migraciones, el esquema real de MySQL y el contenido de los datos. Ninguna tabla, dato, migración, modelo o configuración fue modificada durante este análisis; las recomendaciones son solo propuestas.